# The Architecture of Intent: A Framework for Designing Delegated Systems

**Marcel Aldecoa** *Independent practitioner*

**Paper status:** Skeleton draft (paper v0.1). Position/framework paper, ~12–15 pages target. Companion artifact to the book of the same title.

**Framework version:** v1.3.0 (2026-05-10). Both this paper and the companion book reflect the same framework version; load-bearing commitments (the five archetypes, the four dimensions, the seven failure categories, the four oversight models, the four signal metrics, the four activities, composition as a first-class design surface) move together. See `CHANGELOG.md` at the repository root for the versioning convention and release history.

**Target venue:** arXiv (cs.SE / cs.AI cross-listing). Workshop or journal submission to be decided after first arXiv version is released.

---

## Abstract

Delegated systems — software, organizations, automated pipelines, and increasingly AI agent systems — share a structural problem: humans must express intent precisely enough that a non-human executor can act on it without supervisory rescue. We present *The Architecture of Intent*, a framework that treats intent as a primary design artifact distinct from implementation, and operationalizes it through four load-bearing elements. **Archetypes** commit a system to one of five canonical delegation shapes (Advisor, Executor, Guardian, Synthesizer, Orchestrator) before design begins. **Four orthogonal calibration dimensions** — agency, autonomy, responsibility, reversibility — refactor Shavit & Agarwal's (Shavit, Agarwal, et al. 2023) operational variables into independent levers, decomposing what SAE J3016 (SAE International 2021) packs into a single "automation level". A **fix-locus failure taxonomy** (Cat 1–7) partitions failures by which artifact must change, complementing Cemri et al.'s (Cemri et al. 2025) empirical symptom-locus taxonomy (MAST). **Spec-Driven Development** (GitHub 2024) is the executable protocol that binds the four elements into a specification an agent can run and a human can validate. We instantiate the framework against AI agent systems and demonstrate composition with Microsoft DevSquad Copilot's eight-phase agentic development lifecycle (Microsoft 2026). Three contributions are explicitly novel: the orthogonality operationalization of agency and autonomy, the fix-locus framing of the failure taxonomy, and a new category — **Cat 7 (Perceptual Failure)** — for perceiving-then-acting systems (computer-use agents, browser-use agents) that prior taxonomies do not cover. We position the work as a position-and-framework paper without empirical validation at scale, and argue the framework's reach extends beyond AI agent systems to delegated systems generally.

**Keywords:** intent engineering, agentic development lifecycle, spec-driven development, agent governance, multi-agent systems, AI safety, software architecture.

---

## 1. Introduction

### 1.1 The rising delegation curve

Software has always been a delegation. The programmer delegates a computation to a machine; the team delegates production behavior to the codebase; the organization delegates routine decisions to the systems it has built. What changes in 2024–2026 is the *width* of the delegation: a single human action — writing a brief specification, opening a ticket, asking a question — is increasingly enough to commit a system to a long sequence of consequential decisions made by an executor that is not human and that, even when equipped with explicit clarification APIs, exhibits a documented *execution-first bias* — defaulting to confident continuation rather than to interactive escalation when the spec is ambiguous.

It is worth being precise about this, because the paper's claim is structural and a casual reading might mistake it for a capability claim. By 2026, frontier coding agents and assistants ship with first-class clarification mechanisms: Anthropic's Claude Code exposes an `AskUserQuestion` tool that the model can invoke to pause and request guidance (Anthropic 2024--2025); GitHub Copilot, Cursor, and Cline support multi-turn interactive sessions where the model can stop and ask. The capability is present. What is unreliable is the *judgment* about when to invoke it. Emerging 2026 evaluations of help-seeking behavior in agents — extending standard benchmarks to ambiguous-task variants — find that frontier models perform well on fully specified tasks but collapse on tasks that *should* trigger clarification, instead filling the gap with overconfident assumptions and proceeding to act on wrong beliefs. The root cause is not architectural: reinforcement-learning-from-human-feedback (RLHF) training rewards task *completion*, which produces an inherent asymmetry — pausing to ask is a low-frequency behavior in the reward signal, while one-shotting is the high-probability default. The agent has the tool; the agent does not reliably use it.

Three forms of delegation now coexist with rising authority. Procedural delegation, in which an organization codifies a decision into policy that humans execute, is the oldest. Mechanical delegation, in which deterministic code or controllers execute a fixed program against a defined input domain, is the standard form of software since the 1950s. Agentic delegation, in which an LLM-driven system makes judgment-laden choices over a domain wider than the spec author can enumerate, is the newest and the one whose behavior most departs from the prior two. SAE J3016 (SAE International 2021) formalized graduated delegation for driving automation in 2021, identifying six levels at which a vehicle may handle perception, decision, and action without a human between them; the framing transfers to software systems with one important difference. A vehicle's delegation level is constrained by physics — perception must precede action by milliseconds; intervention windows are short; reversibility approaches zero — and the levels move together. A software system's delegation has no such constraint, and the design space is correspondingly larger and easier to misconfigure.

The cost of imprecise intent in each delegation form has been historically absorbed by human implementers. A poorly written policy was repaired in real time by the operator who knew what was meant. A poorly written spec was repaired by the engineer who had built the previous one. The repair was invisible: it produced a working system whose specification was never quite the artifact that governed its behavior, but that mismatch did not surface as a failure because the human implementer was the bridge. This paper's working hypothesis is that the bridge is now automating, and the cost of imprecise intent — until recently a hidden subsidy paid by professional judgment — is becoming an explicit cost paid in agent rework, scope drift, and post-incident churn. The framework that follows is offered as the structural response: design intent as a primary artifact rather than as the implicit ground of implementation.

### 1.2 Two motivating observations

The framework rests on two observations that have been remarked on individually in practitioner literature (Anthropic 2024a; Shavit, Agarwal, et al. 2023) but that we treat together because they form the gap the framework addresses.

**Observation 1: Human professionals tolerated imprecise intent because they exercised silent judgment to bridge it.** Every senior engineer reading an underspecified ticket has, at some point, supplied missing constraints from experience ("we never delete records that are linked to invoices, so I'll add a check"), escalated ambiguity rather than executing it ("does 'process all orders' include cancelled ones? I'll ask before coding"), and rewritten what was asked into what was clearly meant ("the ticket says 'fix the bug' but the actual fix is a refactor"). None of this judgment was visible to the specification document, which retained its under-specified form because the implementer absorbed the gap. None of it transfers to an automated executor that interprets the document literally. The judgment is real engineering work; it has always been; it has merely been hidden in implementation rather than codified in specification.

**Observation 2: Automated executors make imprecise intent immediately visible as wrong outputs.** Where a senior engineer would have surfaced an ambiguity, an LLM agent — even one with an `AskUserQuestion` tool wired into its harness (Anthropic 2024--2025) — typically does not pause. It resolves the ambiguity probabilistically against the latent distribution of similar cases in its training data, then commits the resolution as a silent assumption and proceeds. This is the *judgment gap*: the technical capability to ask is present, but the calibrated judgment about *when* to ask is not. The resolution the agent picks is rarely the one the spec author wanted, but it is rarely random; it is the *most-probable* resolution given the prompt's surface features, which is a stable phenomenon and therefore a debuggable one. The failure mode is not "the agent is broken" and not "the agent lacks the means to escalate"; the failure mode is "the agent is doing exactly what the spec said, and the spec was wrong, and the agent did not ask because it was rewarded during training for not asking." The cost per call is small — a slightly off-target output, a near-miss at the constraint boundary — but the cost compounds across a deployment, and the post-incident reconstruction is hard precisely because no single call was visibly wrong.

These two observations together produce the framework's central claim: the gap between intent and implementation has not grown; it has merely become *visible*. Spec authors in 2026 are paying explicit costs for what spec authors in 2006 paid implicit costs for. The discipline the framework proposes — naming intent as a designed artifact, shaping it via archetypes, calibrating it along orthogonal dimensions, diagnosing its failures via fix-locus categories, expressing it through Spec-Driven Development — is the discipline that makes the intent layer thick enough to be governable when the implementation layer is automated.

### 1.3 Contribution

This paper contributes a framework for designing delegated systems by treating intent as a primary design artifact distinct from implementation. The framework has four load-bearing elements (introduced in §3, instantiated against AI agent systems in §4): (i) a five-archetype taxonomy committing the system to a delegation shape before design begins; (ii) a four-dimension calibration of agency, autonomy, responsibility, and reversibility, with an explicit *orthogonality argument* extending Shavit & Agarwal's (Shavit, Agarwal, et al. 2023) operational variables; (iii) a *fix-locus* failure taxonomy of seven categories partitioning failures by the artifact whose modification prevents recurrence, and complementing Cemri et al.'s (Cemri et al. 2025) empirical multi-agent symptom partition

(MAST); and (iv) Spec-Driven Development (GitHub 2024) as the executable protocol layer. The framework's primary worked instance is AI agent systems, and §4 demonstrates clean composition with Microsoft DevSquad Copilot's eight-phase agentic development lifecycle (Microsoft 2026).

The paper's *novel* contributions, narrowly framed, are three: the operationalization of autonomy and agency as orthogonal axes (§3.3); the fix-locus framing of the failure taxonomy (§3.4); and **Cat 7 (Perceptual Failure)** as a new category for *perceiving-then-acting systems* (computer-use agents, browser-use agents, robotic systems) that prior taxonomies do not cover, with four subcategories (misidentification, missed element, hallucinated element, state miscount) and structural fixes for each. The paper does *not* claim novelty for SDD as a discipline (lineage from spec-kit and DevSquad), for archetypes as a concept (lineage from Anthropic's *Building Effective Agents* (Anthropic 2024a)), for the four dimensions individually (lineage from SAE J3016 and Shavit & Agarwal), or for Cat 1–6 as categories (synthesis from common practice). The synthesis itself is the larger contribution: the four elements bind into a framework with consistent vocabulary that practitioners can apply in design conversations, in spec reviews, and in post-incident diagnosis.

We position this paper as a *position-and-framework paper* without empirical validation at scale. The framework has been applied across the three worked examples in the companion book (Aldecoa 2026) and in deployments those examples are abstracted from, but quantitative validation across many independent deployments remains future work. §6 enumerates this and the framework's other limitations explicitly.

### 1.4 Paper structure

§2 situates the framework against eight bodies of prior work. §3 develops the framework: intent as a design surface (§3.1), the five archetypes (§3.2), the four-dimension calibration (§3.3), the seven-category fix-locus failure taxonomy (§3.4), and Spec-Driven Development as the protocol (§3.5). §4 instantiates the framework against AI agent systems and the agentic development lifecycle exemplified by DevSquad Copilot, with worked subsections for capability boundaries via the Model Context Protocol, coding agents, and computer-use agents. §5 discusses the framework's applicability boundary, its complementarity with MAST and other multi-agent failure work, and its generalization beyond AI agents. §6 names the framework's limitations honestly. §7 concludes. Appendix A maps each section back to the relevant chapter of the companion book.

---

## 2. Prior work and lineage

> *Section purpose: position the framework against the standing literature it builds on. The honest accounting belongs in this section. We organize the lineage by domain.*

### 2.1 Spec-Driven Development

The framework's *Spec-Driven Development* element (§3.5) is direct lineage from two recent projects rather than original work. GitHub's spec-kit (GitHub 2024), released in 2024, operationalizes spec-first development for AI-assistant-augmented teams: the spec precedes the prompt, the agent executes against the spec, and the spec is the artifact that gets reviewed. Microsoft's DevSquad Copilot (Microsoft 2026), released in 2026, integrates spec-driven development into an eight-phase iterative cycle for agentic software delivery, with explicit phase boundaries, ADR-based architectural decision capture, and TDD-first implementation. Both projects assume the framework's

central premise — that automated execution makes spec-first practice non-optional — and operationalize it for working teams. Our paper does not re-derive SDD; it positions SDD as the *protocol layer* on which the archetype, dimension, and failure-taxonomy layers operate (§3.5). The contribution is the substrate framing, not the substrate itself.

### 2.2 Driving automation and graduated delegation

SAE J3016 (SAE International 2021) defines six levels of driving automation as a graduated handoff of operational responsibility from human to vehicle: from L0 (no automation) through L5 (full automation under all conditions). The structure — operational responsibility shifts as automation widens — is the cleanest pre-existing precedent for the framework's four-dimension calibration. We draw the *delegation-ladder* metaphor explicitly from J3016 and acknowledge that the framework's calibration discipline owes a debt to it. The contribution at this layer is decomposition rather than novelty: J3016's single "automation level" packs decision-space, execution-path, accountability-locus, and recoverability into one ordinal axis because driving's underlying physics constrains the four to move together; software systems do not have that physical constraint, and decomposing the single axis into four orthogonal ones (§3.3) is what gives spec authors independent levers. The generalization from driving to non-driving delegated systems is asserted; the four-dimension shape that supports it is the paper's specific extension.

### 2.3 Agent governance

Shavit, Agarwal, and colleagues (Shavit, Agarwal, et al. 2023) (OpenAI, 2023) define seven operational variables for governing agentic AI systems: *ability*, *agency*, *agency type*, *autonomy*, *alignment*, *accountability*, and *authority*. The framework's four-dimension calibration is a refactoring of a subset of these variables into separable, independent axes with explicit operationalization in spec design. Specifically: our *agency* roughly preserves Shavit & Agarwal's agency and ability; our *autonomy* preserves their autonomy with sharper distinction from agency (the two are conflated in much practitioner discussion); our *responsibility* maps to their accountability with the additional explicit decomposition into authorial, operational, and validation sub-loci (§3.3); our *reversibility* is largely absent from their seven and is contributed by the framework. We do not claim novelty for the underlying concepts — Shavit & Agarwal cover the conceptual ground — and we do claim novelty for the operationalization (which dimensions are orthogonal, how they map to spec clauses, how they jointly underwrite or expose specific failure modes).

### 2.4 Agent design

Anthropic's *Building Effective Agents* (Anthropic 2024a) catalogues practical agent shapes — prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, and autonomous agents — that overlap substantially with the framework's five archetypes (§3.2). A routing pattern is a first decision an Orchestrator makes; an evaluator-optimizer pattern is a Guardian composed with an Executor; an orchestrator-workers pattern is what we call an Orchestrator-over-Executors deployment. The shapes are not original to either work; both projects derive from observation of working deployments. The contribution at this layer is reduction-and-canonicalization: the framework's five archetypes are the smallest decision-ready taxonomy that covers the deployment space, with explicit oversight defaults and explicit composition rules. We acknowledge the synthesis as opinionated rather than derived (§3.2) and acknowledge Anthropic's catalogue as the most-direct prior work the synthesis is reduced from.

### 2.5 Multi-agent failure analysis

Cemri et al.'s MAST (Cemri et al. 2025) (2025) is the closest prior work to the framework's failure taxonomy (§3.4). MAST partitions multi-agent system failures into fourteen empirical categories derived from observation of 200+ deployments: symptoms like *task verification failures*, *inter-agent misalignment*, *specification issues*, *tool use errors*. The partition is by *empirical symptom and locus of observation* — what was seen at the failure site. Our Cat 1–7 partitions by *fix locus* — which artifact must change to prevent recurrence. The two cover the same failure space along different axes and are explicitly complementary; §5.3 develops the multi-axis classification framing. The framework does not compete with MAST on the empirical-symptom axis. Cemri et al.'s contribution is a fourteen-category symptom partition derived empirically; ours is a seven-category fix-locus partition derived from working spec-design practice plus the new Cat 7 (Perceptual) addition. Zhang et al.'s hallucination survey (Zhang et al. 2025) (2025) provides finer-grained partition of model-level failures (which we class as Cat 6 in fix-locus terms) into tool-call hallucination, planning hallucination, and instruction-following inconsistency. We cite this work without re-deriving its structure; the framework treats Cat 6 as a single fix-locus category and points readers to Zhang et al. for the model-output-layer sub-classification.

### 2.6 Pattern languages and software architecture

Alexander, Ishikawa, and Silverstein (Alexander, Ishikawa, and Silverstein 1977) establish the pattern-language form: a structured catalogue of context-problem-solution-resulting-context entries that compose into larger design vocabulary. The framework borrows the form for the archetype catalogue (each archetype has context, structural problem, solution, and resulting-deployment posture). We do not, however, claim that the framework constitutes a pattern language in Alexander's strict sense. Alexander's standard requires extensive empirical derivation of patterns from working examples; our archetypes are an opinionated synthesis with limited empirical grounding (three worked examples in the companion book). The framework borrows the *form* and explicitly disclaims the *standard*. Brooks's *The Mythical Man-Month* (Brooks 1975) grounds the essential-versus-accidental-complexity distinction we use in §3.1's argument that intent has been the persistent design surface. Meyer's Design by Contract (Meyer 1992) and Jackson's specifications work (Jackson 1995) anchor the contract-first software-engineering tradition that SDD inherits from. None of these are novel work; they are the load-bearing prior art the framework is operating in continuity with.

### 2.7 Systems thinking and human error

Meadows's *Thinking in Systems* (Meadows 2008) and Reason's *Human Error* (Reason 1990) frame the broader systems-and-human-factors tradition. Reason's distinction between *active failures* (errors at the sharp end of the system, performed by operators) and *latent conditions* (system-level conditions that make the active failures more likely) parallels the framework's distinction between Cat 6 (Model-level, active — the agent confidently produced incorrect content) and Cat 1 (Spec, latent — the spec authorized the wrong action and waited for an executor to act on the authorization). The parallel is suggestive rather than load-bearing for the present paper; we acknowledge the conceptual continuity and refer readers to Reason for the deeper development. The framework's Cat 4 (Oversight) maps onto Reason's defenses-in-depth thinking, where each defensive layer fails for a reason that is itself diagnosable. We do not attempt a re-derivation in the body of the paper; the citation is to acknowledge the framework's place in a broader tradition that did not start with agent systems.

### 2.8 Inference economics and prompt architecture

Pope et al. (Pope et al. 2022) ground the inference-cost economics underlying transformer-based systems: the per-token, per-call, and per-prefix cost shapes that determine which deployments scale and which do not. The framework cites this work for the application section's prompt-caching discussion (which the companion book covers in depth and which the paper references rather than re-derives). Liu et al. (Liu et al. 2023) document the *Lost in the Middle* phenomenon: long-context language models exhibit non-uniform attention across the context window, with content placed in the middle of long contexts attended to less reliably than content at the beginning or end. The empirical observation motivates the context-budget patterns the framework's application chapter recommends — particularly for coding agents operating on whole-repository contexts where critical constraints can be placed where the model attends to them least reliably. Both works are cited for the application section (§4 and the companion book) rather than for the framework section (§3); we include them in §2 because they are part of the prior-work landscape practitioners adopting the framework should know about.

---

## 3. The framework

> *The framework on one page. Figure 1 below shows the four load-bearing elements of this section — archetypes, dimensions, failure categories, Spec-Driven Development — together with the four activities that bind them and the four signal metrics that close the loop. The subsections of §3 develop each element; the figure is meant to be returned to between subsections, not read once.*

### 3.1 Intent as a design surface

We define **intent** as the human-authored description of what a delegated system should do, the constraints it must respect, and the conditions that distinguish acceptable from unacceptable outcomes. Intent is a designed artifact, separate from three things it is commonly conflated with: implementation, requirements, and policy.

**Intent versus implementation.** Implementation is what the executor produces — code, an action, a refactored document. Intent is what the executor was supposed to produce. The distinction is foundational and chronically collapsed in practice (Brooks 1975). Software-engineering literature has named the collapse for decades — Brooks's distinction between *essential* and *accidental* complexity, Jackson's separation of *requirements problem* from *machine* (Jackson 1995), Meyer's contract-first discipline (Meyer 1992) — but the collapse has been tolerable because human implementers absorbed the gap silently. A senior engineer reading a one-line ticket supplied missing constraints from experience, escalated ambiguity rather than executing it, and rewrote what was asked into what was clearly meant. None of that judgment was visible in the specification document. An automated executor inherits the technical surface to do some of this — modern coding agents and assistants ship with interactive clarification APIs (Anthropic 2024--2025) — but inherits neither the calibrated judgment about *when* to escalate nor the training reward signal that would make escalation default. When code becomes the executor, the gap between intent and implementation shows up as bugs; when an LLM agent becomes the executor, the gap shows up as outputs that are syntactically reasonable, individually defensible, and collectively wrong.

**Intent versus requirements.** Requirements describe what stakeholders ask for. Intent is what

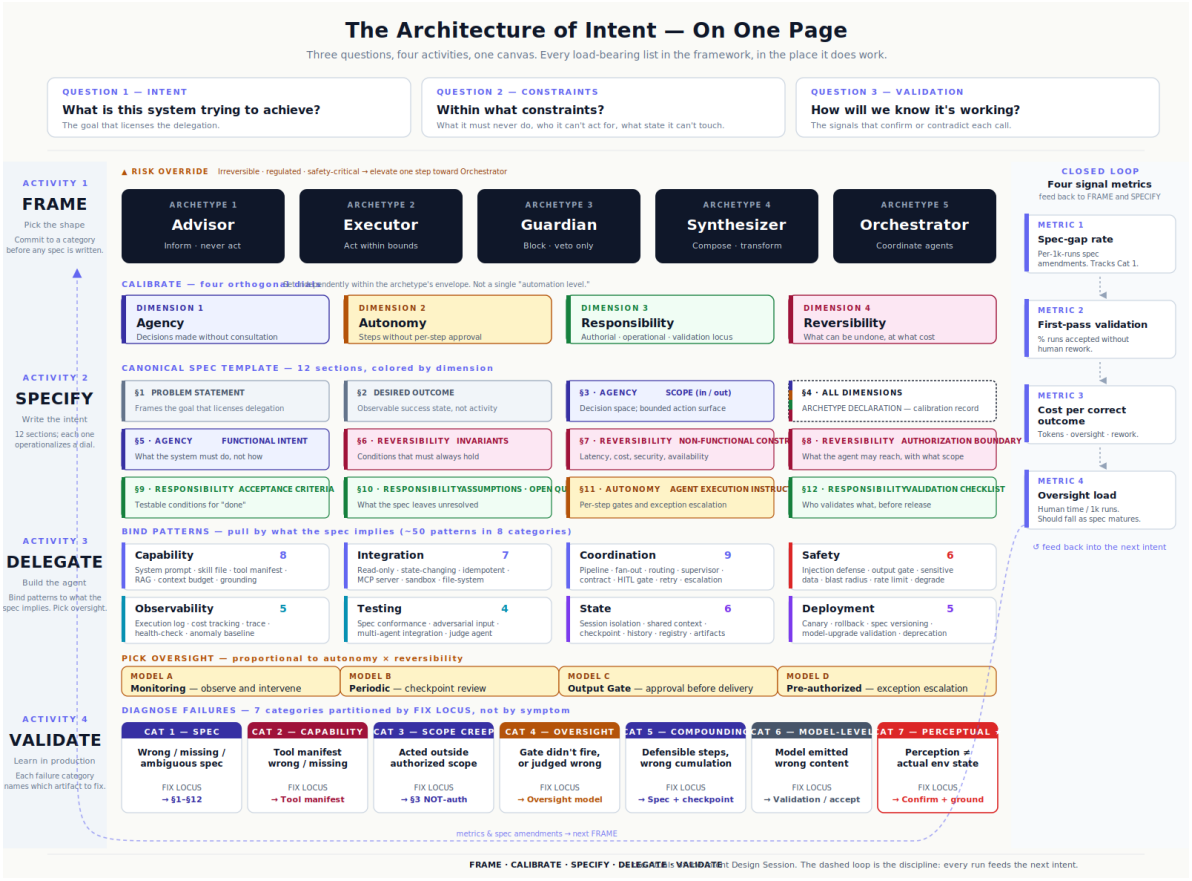


Figure 1: **Figure 1.** The Architecture of Intent on one page. Three questions every delegated system answers (top); four activities that work them out (Frame · Specify · Delegate · Validate, on the spine); the load-bearing constructs each activity binds — five archetypes, four orthogonal calibration dimensions, twelve canonical spec sections (each colored by the dimension it operationalizes), eight pattern categories, four oversight models, seven fix-locus failure categories; and four signal metrics on the right rail that feed back into the next intent. §3.2–§3.5 develop the constructs; §4 instantiates them against AI agent systems.

the system author decides to build. The two diverge whenever the request is incomplete (most cases), contradictory (common), or wrong (occasional). A traditional requirements document captures the request; an intent specification captures the synthesized design decision after the request has been interrogated, prioritized, and reduced to a coherent set of behaviors and constraints. The distinction matters because requirements churn and intent must not. A spec that re-litigates requirements every time a stakeholder changes their mind cannot be the control surface an agent executes against; a spec that resists requirement churn by holding the design decision constant is what the framework calls intent.

**Intent versus policy.** Policy is organizational or regulatory — what the company, the law, or the ethics review board requires across all systems. Intent is system-specific. Policy and intent compose: policy provides cross-cutting constraints that every spec inherits; intent provides the per-system behavior the spec describes within those constraints. Conflating the two produces specifications that treat compliance as a feature (causing compliance to drift with system priorities) or treat behavior as compliance (causing organizations to over-formalize routine engineering decisions). The framework holds them apart: policy belongs in a constitutional layer above the spec; intent belongs in the spec.

**Intent as the locus of design.** Once these three distinctions are in place, intent is no longer an implicit shadow of requirements or a soft-touch precursor to code. Intent is the artifact the system author actually designs. The five-archetype framework (§3.2) provides shape; the four-dimension calibration (§3.3) provides axes; the failure taxonomy (§3.4) tells the author where to look when execution diverges from intent; Spec-Driven Development (§3.5) is the protocol that makes intent executable and verifiable. Each subsequent section operates on the assumption that intent is something deliberately authored, not something inferred after the fact.

**Why this matters now.** Three observations make the design-surface framing acute in 2026 in a way it was not in 1995. First, the executor is increasingly an LLM agent rather than a deterministic compiler, and the LLM agent — for the training-incentive reasons established in §1.1 and §1.2 — defaults to one-shotting through ambiguity rather than to escalating it, *despite* having the technical means to escalate. Second, the rate of delegation is faster: an agent loop completes a sequence of decisions in seconds where a human team would have completed it in days, with the consequence that the cost of imprecise intent is amortized across more decisions per unit time. Third, the locus of design effort has shifted: a software engineer spending an hour on a tightly-bounded spec produces work that an agent loop can extend into hundreds of correct outputs; the same hour spent on the implementation directly produces a single output. The leverage on intent has gone up because the executor's throughput has gone up. The collapse of intent into implementation that human professionals absorbed for decades is no longer free.

The rest of this section operates on intent as a designed artifact. Section 3.2 names five canonical shapes that intent takes; section 3.3 names four dimensions that calibrate it; section 3.4 names seven categories of failure that diagnose where intent broke; section 3.5 names the protocol that makes intent executable.

### 3.2 Archetypes

We propose five canonical archetypes — *Advisor*, *Executor*, *Guardian*, *Synthesizer*, *Orchestrator* — as the smallest decision-ready taxonomy that covers the delegation shapes practitioners actually deploy. Each archetype is a pre-commitment to a behavioral envelope: a posture toward action, a relationship to oversight, a default risk profile, and a set of invariants that follow from the shape.

Choosing the archetype is the first design decision an intent author makes, and it constrains every subsequent decision.

**The five archetypes are summarized below.**

| Archetype | Core function | Agency level | Risk posture | Default oversight |
|---|---|---|---|---|
| **Advisor** | Surface information, options, recommendations; never act on the world | Minimal | Low | Human decides and acts |
| **Executor** | Carry out well-defined tasks autonomously within strict bounds | High | Medium | Pre-approved scope; exception-based escalation |
| **Guardian** | Enforce rules, validate integrity, block constraint violations | Low (veto only) | Low | Alerts; humans resolve |
| **Synthesizer** | Aggregate, distill, or compose from multiple sources into a coherent artifact | Moderate | Medium | Output review above threshold |
| **Orchestrator** | Coordinate multiple agents or services toward a compound goal | High | High | Active oversight; escalation paths required |

**The selection tree.** Figure 2 resolves archetype selection in four sequential questions, applied in order, stopping at the first match. The questions are asked in increasing decision difficulty: the first concerns externally observable behavior (does the system act on the world without a human between its output and the consequence?); the second concerns purpose (is the primary function protective rather than productive?); the third concerns relationship to other systems (does the work involve coordinating other agents?); the fourth concerns output shape (is the primary product a synthesized artifact?). The default after all four questions is *Executor*, which captures most well-bounded autonomous-action systems.

A *risk override* sits above the tree's output. If the system's actions touch high-consequence domains (irreversible state changes, regulated data, safety-critical control), the assigned archetype is elevated to the next-most-restrictive in the path *Advisor* < *Guardian* < *Synthesizer* < *Executor* < *Orchestrator*. The override mechanism encodes the principle that the cost of misclassification is asymmetric: classifying an Executor as an Advisor produces an under-deployed system; classifying an Advisor as an Executor produces a deployment that exceeds its authorization. The override defaults to the conservative direction.

**Composition.** A single deployment frequently hosts more than one archetype. An Orchestrator coordinating Executors, with a Guardian validating each Executor's output before downstream effects, is a common shape in production agent systems. The framework treats composition as the rule, not the exception: each archetype is a per-component classification, not a per-deployment one. Composition's main constraint is that Guardian archetypes — which are protective rather than productive — must sit on the boundary of every productive archetype's effect surface, not in series after it. A Guardian downstream of an Executor's irreversible action is a check that runs after the damage; a Guardian gating the Executor's action is a check that runs before.

**Pre-commitment, not classification.** A practical implication of the archetype framing is that archetype is *committed before* design rather than *inferred after*. A spec author who sits down to design a customer-support system does not write the spec and then ask "is this an Advisor or an Executor?" — that conflates intent with implementation in the way §3.1 forbade. They commit to the archetype first (informed by Figure 2 and the risk override), then design the system within its envelope. The pre-commitment makes downstream design decisions traceable: oversight model is determined by archetype; capability boundary is determined by archetype; invariants follow from archetype. When a deployment exhibits incoherent oversight ("the agent is allowed to do X but the team reviews every output anyway"), the diagnosis is almost always that the archetype was committed implicitly rather than explicitly, and downstream choices were made against a different shape than the one declared.

**Relation to prior work.** The five archetypes are an opinionated synthesis, not an original taxonomy. Anthropic's *Building Effective Agents* (Anthropic 2024a) catalogues practical agent shapes — prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, autonomous agents — that overlap with our five along multiple cuts. A routing pattern is what we call an Orchestrator's first decision; an evaluator-optimizer pattern is a Guardian composed with an Executor; an orchestrator-workers pattern is what we call Orchestrator-over-Executors. The contribution is not that these shapes are new; it is that they are reduced to a smaller set of decision-ready categories, each with a defined oversight default and risk posture, indexed by a four-question decision tree. Practitioners report — in a way the present paper cannot empirically verify — that the smaller, opinionated taxonomy is easier to commit to in design conversations than the larger catalogue is.

**Why five.** The five-cardinality choice is a working decision rather than a derived one. Three would force a coarser commitment than the design space supports (Advisor and Executor would have to absorb Synthesizer's distinct shape; Guardian would have to absorb Executor's). Seven or nine would re-fragment the categories that the five-cardinality holds together. The framework offers five as an opinionated default; practitioners are free to refine the catalogue per their domain. What the framework does not permit is operating without an archetype commitment at all.

**Where the taxonomy is under pressure, and why composition is the answer.** Three classes of system, all increasingly common as of 2026, do not fit any single archetype. *Coding agents* (Cursor, Cline, Devin, Claude Code) move within one session between Synthesizer mode (planning, summarizing) and Executor mode (writing files, running tests, opening PRs), and increasingly into Orchestrator-over-self mode for harder tasks. *Deep-research agents* synthesize a final report and orchestrate sub-research recursively. *Self-improving / training agents* have a primary act ("train") that is none of *inform*, *execute*, *enforce*, *synthesize*, or *coordinate*. The framework's commitment is that **composition is a first-class design surface** — one governing archetype, embedded components or modes, declared transitions, cross-mode invariants — rather than that the taxonomy needs a sixth archetype. Adding a sixth would have to name a primary act not on the list of five; coding agents and deep-research agents do not have a sixth primary act, they have

several of the existing five used in sequence within a session. Naming the absence of a primary act ("operates in multiple modes") would re-name composition while losing the structural clarity of declared transitions. Self-improving agents are honestly two-system deployments — a meta-system (Synthesizer over the inner agent's outputs) and an inner agent — with a clean handoff, not one-system compositions. The framework remains open to extension: a genuine sixth archetype must demonstrate a *governance profile* (agency level, oversight model, reversibility posture, invariant set) that no composition of the existing five provides. As of 2026, no class meets that bar; composition does the work cleanly. The companion book (Aldecoa 2026) develops the structural surface in full, including a *Composition Declaration* sub-template fragment for §4 of the canonical spec and a *mode-switching* pattern (Pattern E) that handles the three pressure-point classes above directly.

**Figure 2** (below) is the working version of the selection tree. It is meant to be used in design sessions, in spec reviews, and in onboarding new engineers to the framework — not to be read once and remembered.

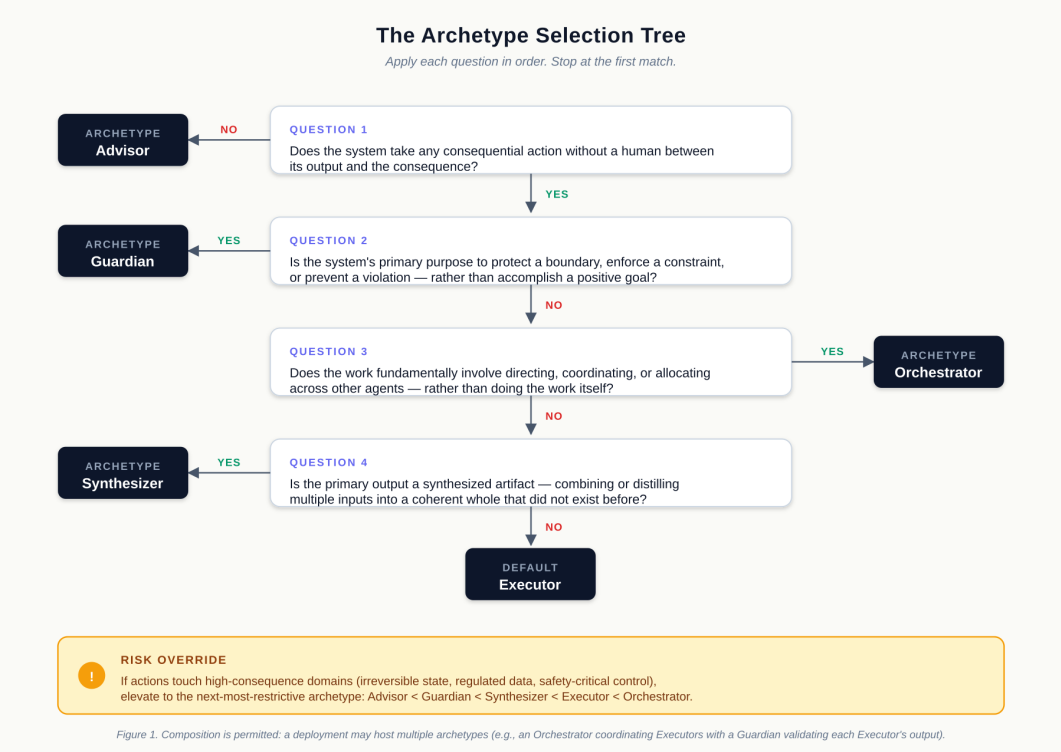


Figure 2: **Figure 2.** The archetype selection tree. Apply questions in order; stop at the first match. The risk-override sidebar elevates the assigned archetype when the system's actions touch high-consequence domains. Composition is permitted: a single deployment may host multiple archetypes.

### 3.3 Four dimensions of calibration

The archetype framing of §3.2 commits a system to a delegation shape. The four-dimension calibration framing of this subsection commits the system, *within that shape*, to specific levels of independence, judgment latitude, accountability locus, and recoverability. The four dimensions are **agency**, **autonomy**, **responsibility**, and **reversibility**. The central claim of this subsection is that these four are *orthogonal* — independent in principle and in practice — and that collapsing

them into a single "automation level," as SAE J3016 (SAE International 2021) does for driving, loses design space that practitioners need.

**Agency.** The capacity to choose actions in pursuit of a goal. Wider agency means more decisions the system makes without consulting a supervisor. A code-review Advisor exercises narrow agency: it produces recommendations, and the choice of which recommendations to act on belongs to the human reviewer. An autonomous coding agent like Devin exercises wide agency: it chooses what to refactor, in what order, and against which tests, without consulting the human between decisions. Agency is a property of the system's *decision space* — what the system is permitted to choose.

**Autonomy.** Operational independence from per-step human authorization. Wider autonomy means more operations execute without an approval gate between the system's decision and its effect. A deterministic CI/CD pipeline exercises wide autonomy: every step runs without a human approval. A compliance-review system that surfaces every potential violation for human resolution exercises narrow autonomy: every operation gates on a human. Autonomy is a property of the system's *execution path* — what the system is permitted to execute without supervisory approval at the step level.

**Responsibility.** The locus of accountability for outcomes. Distinct from agency (decision space) and autonomy (execution path), responsibility names *who is on the hook* when things go wrong. The framework distinguishes three sub-loci: *authorial* (who specified the system), *operational* (who is executing it on this run), and *validation* (who validated that this run's output should be accepted). In well-designed systems all three are explicit; in poorly-designed systems responsibility is diffuse, which is the underlying condition that produces both Cat 4 (Oversight) failures and the post-incident finger-pointing that diffuse responsibility enables.

**Reversibility.** The capacity to undo or recover from an incorrect action. Reversibility is a property of the *system's environment and tooling*, not of the system's intent. Soft deletes, draft queues, audit logs that support replay, undo buffers, and approval gates are reversibility-extending mechanisms. Sending an email, processing a payment, deleting a record without an audit trail, and most physical-world actions are reversibility-collapsing. A spec author who treats reversibility as a fixed environmental property is missing a lever; a spec author who recognizes that reversibility can be *designed* — by routing irreversible actions through a draft-and-confirm pattern, by adding a reversal endpoint to a state-changing API, or by adding an approval gate before high-consequence operations — has more dimensions to calibrate.

**The orthogonality argument.** These four dimensions are independent. Figure 3 plots two of the four (Agency × Autonomy) against each other and shows that all four quadrants contain real deployments — meaning the dimensions are not dependent variables that cluster along a diagonal. A system can have wide agency but narrow autonomy: a compliance Guardian with explicit gates exercises high decision latitude per call (it judges whether a transaction is suspicious) but every call surfaces for human approval (no per-step automation). A system can have narrow agency but wide autonomy: a deterministic CI/CD pipeline exercises no judgment per step (the steps are fully prescribed) but executes the entire sequence without per-step approval (no gates). The other two pairs are similarly orthogonal: a system with wide responsibility (clear authorial ownership) and narrow reversibility (every action is permanent) is a high-stakes Executor; a system with diffuse responsibility (no clear owner) and wide reversibility (everything is a draft) is what most undisciplined agent prototypes converge to.

**Why the orthogonality matters in practice.** When the four dimensions are treated as a single "automation level," spec authors have one lever to pull. The lever has six positions in the

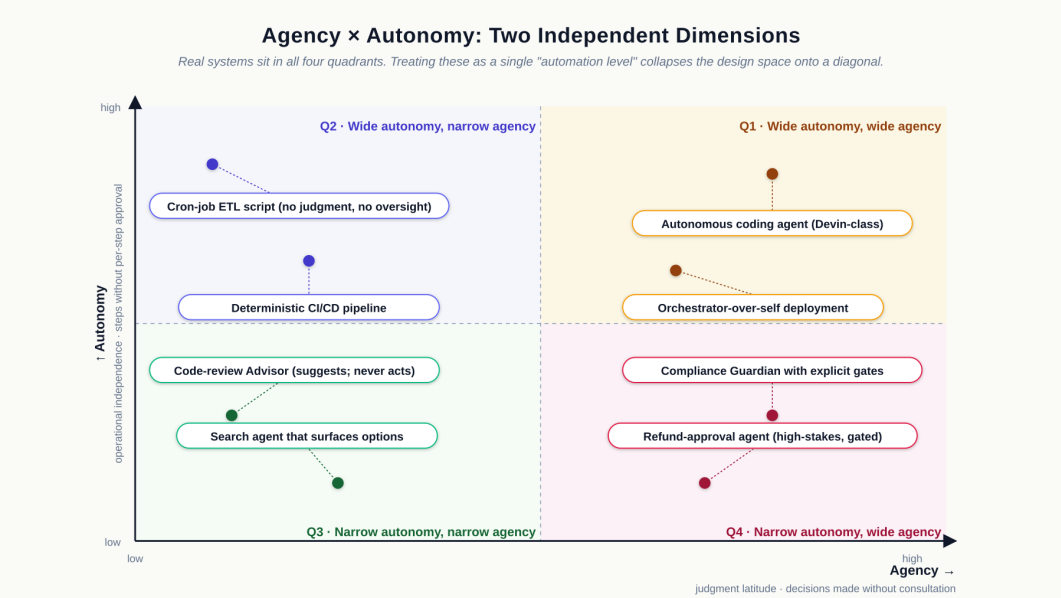


Figure 3: **Figure 3.** Two of the four calibration dimensions plotted against each other. Each quadrant has real deployments; no quadrant is empty. The other two dimensions (Responsibility, Reversibility) are similarly orthogonal. Treating Agency and Autonomy as a single "automation level" — as SAE J3016 does for driving (SAE International 2021) — collapses this design space onto a diagonal.

SAE J3016 case (SAE International 2021) and seven operational variables in Shavit & Agarwal (Shavit, Agarwal, et al. 2023); in either case, the positions are correlated. Pulling the lever toward more automation moves all aspects of the system together — more decisions, fewer gates, more diffuse accountability, less recoverability. This is fine for driving automation, where the underlying physics constrains the dimensions to move together (a vehicle that exercises wide agency needs wide autonomy because human reaction time cannot insert itself between perception and steering); it is wrong for software-system design, where the dimensions are independently controllable and the design space is much larger than a single ladder permits. The framework's contribution at this layer is the *operationalization of the orthogonality*, not the discovery of the dimensions individually — Shavit & Agarwal's seven variables already covered the conceptual ground.

**Mapping to spec clauses.** Each dimension maps to specific clauses in a Spec-Driven Development specification. Agency maps to *§3 (Authorized Scope)* and *§4 (NOT-Authorized Scope)* — the spec clauses that bound the agent's decision space. Autonomy maps to *§4 (Oversight Model)* — the clauses that declare which steps gate on human approval and which run automatically. Responsibility maps to *§1 (Owner)* and *§12 (Validation Checklist)* — the clauses that assign authorial, operational, and validation accountability explicitly. Reversibility maps to *§7 (Tool Manifest)* and *§8 (Authorization Boundary)* — the clauses that distinguish reversible from irreversible tool effects and constrain the latter behind explicit gates. The mapping is not novel to this paper; what the framework adds is the *insistence* that all four mappings be present in every spec, with explicit values, rather than left implicit.

**A worked configuration: a low-agency Guardian.** The compliance Guardian on a financial-transactions platform: agency is *narrow* (the system's decision space is a single classification — flag or pass); autonomy is *narrow* (every flag surfaces for human resolution); responsibility is *clear* (compliance officer authors the spec, the operations team executes, the same compliance officer validates each flag); reversibility is *high* (a flagged-but-not-yet-confirmed transaction is reversible by definition; a passed transaction proceeds, but the system's effect was only the binary flag, which is itself reversible by re-flagging). Net: a tightly-bounded system with high oversight, high recoverability, and concentrated accountability — the right calibration for the domain.

**A worked configuration: a high-agency Executor.** A coding agent (in-loop, on a real repository, mid-sized scope): agency is *wide* (the system chooses what files to modify, what abstractions to introduce, which tests to extend); autonomy is *wide* per task but gated at PR boundaries (no per-step approval, but every set of changes surfaces as a PR for human merge); responsibility is *distributed* (the human ticket author is the authorial owner; the agent is the operational executor; the human merging the PR is the validator); reversibility is *moderate* (uncommitted changes are fully reversible; committed-but-unmerged changes are reversible by reset; merged changes require a revert). Net: a system that converts a single human task ticket into a substantial body of work, with the gate at the boundary where reversibility is still cheap.

The four dimensions, used as design levers, generate spec configurations that match the deployment's risk profile rather than its taxonomy label. The Guardian and the coding agent above are different archetypes, but the dimensional analysis is the same conceptual procedure: name the value on each axis, justify each value, write the spec clauses that operationalize each value.

### 3.4 The fix-locus failure taxonomy (Cat 1–7)

When a delegated system produces a wrong outcome, the operationally useful question is not *what failed* but *which artifact has to change so this doesn't happen again.* We propose a seven-category

partition by *fix locus* — the artifact whose modification prevents the failure's recurrence. The first six categories synthesize common practice. The seventh, *Perceptual Failure*, is novel to this work and addresses a class of failure that emerges in *perceiving-then-acting* systems (computer-use agents, browser-use agents, robotic systems) which prior taxonomies do not cover.

**The seven categories.**

| Category | Failure shape | Fix locus |
|---|---|---|
| Cat 1 — Spec | Spec said the wrong thing, omitted a needed clause, or left an ambiguity the executor resolved against the author's intent | The spec |
| Cat 2 — Capability | Capability boundary was wrong: tool manifest exposed a tool that should have been restricted, lacked a tool the task required, or scoped a tool too broadly | Tool manifest; capability authorization |
| Cat 3 — Scope creep | Agent acted outside the authorized scope, often by interpreting the spec's positive clauses without enforcing its NOT-authorized clauses | Spec NOT-authorized clauses; agent system prompt |
| Cat 4 — Oversight | A needed oversight gate did not fire, or the gate fired but the human reviewer's judgment was misaligned with the spec's criteria | Oversight model; gate configuration; reviewer training |
| Cat 5 — Compounding | A chain of individually defensible steps produced an incorrect cumulative outcome; no single step was wrong, the composition was | System spec; checkpoint pattern; evaluator-optimizer composition |
| Cat 6 — Model-level | The model produced confidently incorrect content despite a correct spec, a correct manifest, and intact oversight; the failure is at the underlying capability layer | Structural validation; allowlist resolution; accept residual risk |
| **Cat 7 — Perceptual** | **A perceiving-then-acting system's perception did not match the actual state of the environment; the system acted on a wrong model of reality** | **Confirmation gate; screenshot-then-verify; multimodal grounding; element-allowlist** |

**The fix-locus framing.** A failure's *symptom* is what was observed (a wrong refund amount; a deleted test; a clicked button on the wrong domain). A failure's *fix locus* is the artifact whose modification eliminates the failure's recurrence. The two are not the same and frequently diverge. A wrong refund amount may have been observed in production (symptom), but the spec was the locus that authorized the wrong amount (fix locus: spec, Cat 1). A deleted test was observed in CI (symptom), but the spec did not forbid test deletion (fix locus: spec, Cat 1; or capability, Cat 2, if the agent should not have had write access to the test directory). The fix-locus framing is what makes the taxonomy operationally useful: it tells the team *which artifact to update*, which is what the post-incident response actually requires.

**Cat 7 — Perceptual Failure: the novel category.** Computer-use agents (Anthropic 2024c; OpenAI 2025; Google 2025) perceive a screen via vision and act via simulated input. Their failures include shapes that prior taxonomies, including MAST (Cemri et al. 2025) and the hallucination survey of Zhang et al. (Zhang et al. 2025), do not partition: the system's perception of the environment diverges from the environment's actual state, and the system acts on the wrong perception. We name this *Perceptual Failure* and identify four sub-categories:

- **Misidentification.** The system identifies an interface element correctly as a category (e.g., "a button") but assigns it the wrong role (e.g., a "Cancel" button identified as "Confirm"). The fix is structural: a confirmation gate before high-consequence actions, where the gate's prompt is generated from the system's claimed intent ("you are about to click Confirm — is that what you mean?") and surfaces the discrepancy to a human reviewer or a Guardian.
- **Missed element.** The system fails to perceive an element that was present and material to the action (a modal dialog, an error toast, a form-validation message). The fix is structural: screenshot-then-verify before proceeding, where the verification step asserts the expected state of the screen against the system's plan and halts on mismatch.
- **Hallucinated element.** The system perceives an element that was not present (a button it expected to see, a list item it expected to find), and acts on the perception. The fix is element-allowlist plus DOM-grounded verification where DOM is available; for canvas-rendered or image-only environments, the fix is reduced reliability and required human checkpoints.
- **State miscount.** The system asserts a position-based fact about a list, scroll position, or sequence that was true at one moment but no longer true at the moment the system acted on it (e.g., "the third item" became the second after a scroll). The fix is re-verification of position-based facts at the moment of use, not at the moment of perception.

These four sub-categories are not exhaustive — perception failure modes will continue to be discovered as more computer-use deployments surface them — but they cover the failure shapes practitioners report most often as of 2026 and are sufficient for the category to be operationally useful in spec design and in red-team protocols.

**Why Cat 7 is necessary.** No prior taxonomy partitions perceiving-then-acting failure as a distinct class. MAST (Cemri et al. 2025) is empirically derived from text-based multi-agent systems and does not specifically address vision-language perception failure modes. The hallucination survey of Zhang et al. (Zhang et al. 2025) partitions hallucination at the model output layer (tool-call, planning, instruction-following) rather than at the perception-action interface. OWASP LLM Top 10 (OWASP Foundation 2025) addresses prompt-injection categories (LLM01 multimodal sub-category) and improper output handling (LLM05) but treats perception failure as a downstream consequence rather than a first-class failure category. The result is that practitioners deploying computer-use agents in 2026 lack a taxonomy slot for the failures they actually observe, and the fixes (confirmation gates, screenshot-verification, element-allowlists) get reinvented per deployment.

Cat 7 is the smallest contribution that gives the taxonomy a slot.

**Composition with prior work.** The fix-locus framing complements rather than replaces the symptom-locus framings of prior work. MAST tells the team *what failed at the symptom layer*: the agent emitted an incorrect tool call, the supervisor failed to verify, the handoff dropped state. The Cat 1–7 framing tells the team *which artifact must change*: the spec, the manifest, the oversight model, the structural validation, the perception-verification step. Together, they form a two-axis classification: a finding's MAST category (what failed) and its Cat 1–7 (where to fix it). A single MAST finding may map to multiple Cat 1–7 categories (a spec gap that allowed a manifest exposure that produced an incorrect tool call is Cat 1 + Cat 2), and a single Cat 1–7 fix may resolve multiple MAST findings (tightening the spec's NOT-authorized clauses is Cat 1 work that closes several MAST symptoms at once).

**The taxonomy's limits.** The seven categories partition the failure space coarsely, which is its operational virtue and its conceptual limit. Reasonable readers may propose six (collapsing Cat 6 and Cat 7 into a single "model-level" with sub-types) or eight (separating *spec gap* from *spec ambiguity* in Cat 1, as the book's living-spec chapter does). The framework's working choice is seven. The contribution is the *fix-locus framing* and the *Cat 7 partition*; the precise cardinality is opinionated rather than derived.

### 3.5 Spec-Driven Development as the protocol

The three previous subsections name what to design (intent), how to shape it (archetypes), and how to calibrate it (dimensions); §3.4 names how to diagnose its failures. None of those by themselves produces an executable artifact. **Spec-Driven Development (SDD)** is the protocol that does — the discipline of writing a complete, validated specification *before* the executor runs, treating the spec as the authoritative source against which output is reviewed, and updating the spec rather than the output when execution reveals the spec was wrong.

SDD as a discipline predates this paper. GitHub's spec-kit (GitHub 2024) operationalizes spec-first practice with AI assistants; Microsoft's DevSquad Copilot (Microsoft 2026) integrates SDD into an eight-phase iterative cycle for agentic software delivery. Neither project originated the idea: the contract-first thread runs through Meyer's Design by Contract (Meyer 1992) and Jackson's specifications work (Jackson 1995); the spec-as-truth thread runs through formal-methods literature decades earlier. What SDD adds, in its 2024–2026 form, is the recognition that *agentic execution makes spec-first non-optional*: a spec that exists only to satisfy a process review, while engineers code from informal understanding, is a documentation artifact; a spec that an agent literally executes against is a control surface. The framework relies on the latter.

**The spec lifecycle.** SDD operates on a five-phase cycle: *intent capture* (the system author articulates what is to be designed), *specification* (the intent becomes a structured document with archetype, scope, constraints, oversight model, acceptance criteria), *execution* (the agent or automated executor produces output against the spec), *validation* (a human or a Guardian validates the output against the spec's acceptance criteria), and *evolution* (the spec is updated when validation reveals the spec, not the output, was wrong). The framework's contribution at this layer is the recognition that the *evolution* phase is where the failure taxonomy of §3.4 operationalizes: a Cat 1 failure produces a spec update; a Cat 2 failure produces a manifest update; a Cat 4 failure produces an oversight-model update; and so on. The taxonomy is the diagnostic that drives the evolution phase.

**SDD as the structural response to the judgment gap.** The execution-first bias documented in §1.1–§1.2 is the strongest argument for SDD as a discipline. If the agent reliably escalated ambiguity at the right moments, a free-form prompt and a clarification API would suffice. Because the agent does *not* reliably escalate — even when the API exists, the model's training rewards completion — the discipline of resolving ambiguity *before* execution must move out of the agent and into the spec lifecycle. The *specification* and *execution* phases become two distinct phases rather than one continuous loop, separated by an explicit clarification step that runs against the human, not the agent. Concretely: GitHub spec-kit's `/speckit.clarify` command (GitHub 2024) surfaces ambiguities for human resolution before the agent is allowed to act. The discipline is structural — the human-in-the-loop is forced upstream of execution rather than relied on to be invoked downstream. SDD is, in this sense, the operational acknowledgment that the agent's help-seeking judgment cannot be trusted to be calibrated; the workflow must compensate.

**The canonical template.** A SDD specification, in the framework's canonical form, contains thirteen sections — problem statement, objective, authorized scope, NOT-authorized scope (with the archetype declared in this same section), tool manifest, invariants, non-functional constraints, acceptance criteria, agent execution instructions, oversight model, validation checklist, spec evolution log, and the gap log. We do not re-derive the template here; the companion book contains the full development. What this paper claims is that the four-dimension calibration of §3.3 maps cleanly to specific clauses in this template (§3 and §4 carry agency; §4's oversight model carries autonomy; §1 and §12 carry responsibility; §7 and §8 carry reversibility), and that the seven failure categories of §3.4 map cleanly to specific update sites in the same template (Cat 1 updates §3, §4, or §6 depending on the gap; Cat 2 updates §7; Cat 4 updates §4's oversight clauses; Cat 5 updates §11 with checkpoint or evaluator-optimizer instructions; Cat 7 updates §11 with verification-before-action requirements). The mapping makes the framework operational: a failure category dictates which clause to update, and the spec evolution log records the change.

**The living-spec discipline.** A spec that never updates is either a spec for a system that has never been used or a spec the team has stopped governing. A spec that updates with every preference change is a spec being abused as a conversation transcript. The living-spec discipline names a middle path: *spec gaps* (Cat 1) and *spec ambiguities* (a sub-class of Cat 1 the book treats separately) trigger spec updates; *implementation failures* (Cat 6, sometimes Cat 2) and *preference changes* (which the framework explicitly excludes from the taxonomy) do not. The discipline is what prevents both spec rot and spec churn. Empirically, teams that adopt SDD without the living-spec discipline degrade within two quarters into either form of failure mode; teams that adopt the discipline maintain spec-as-control-surface over multiple quarters (Microsoft 2026 reports analogous observations).

**Why SDD is the protocol layer rather than a fifth element of the framework.** Archetypes (§3.2), dimensions (§3.3), and the failure taxonomy (§3.4) are *descriptive* — they tell the system author how to think about a delegated system. SDD is *prescriptive* — it tells the author how to write down the result so that an executor can act on it and a validator can check it. The framework treats SDD as a substrate rather than a peer element: archetypes commit at the archetype-declaration clause of an SDD spec; dimensions calibrate at specific other clauses; the taxonomy operates on the spec evolution log. Without SDD as the substrate, the framework's other elements are concepts without artifacts — a vocabulary the team can use in conversation but cannot enforce in code. With SDD, they become the structure of an executable specification.

The framework, in summary, is: *intent as a designed artifact* (§3.1), shaped by *archetypes* (§3.2), calibrated along *four orthogonal dimensions* (§3.3), diagnosed against *seven failure categories* (§3.4), expressed and evolved through *Spec-Driven Development* (§3.5). Section 4 instantiates the frame-

work against AI agent systems as the most-acute current application; §5 discusses generalization beyond agents; §6 names the framework's limitations honestly.

---

## 4. Worked application: AI agent systems

*Section purpose: instantiate the framework against the most-acute current case. The book provides three full worked examples (a customer support multi-agent system, a code-generation pipeline, an in-loop coding agent); the paper summarizes the third because it is the most-deployed agent class of 2024–2026 and the compositional shape is rich.*

### 4.1 The agentic development lifecycle and DevSquad Copilot

The framework presented in §3 is process-agnostic. It commits to *what* must be designed (intent, archetype, dimensions, failure taxonomy) and *how* it must be expressed (Spec-Driven Development), but it does not commit to *when* in the development cadence each artifact is produced. Practitioners need that commitment, and Microsoft DevSquad Copilot (Microsoft 2026) supplies it: an eight-phase iterative cycle for *agentic software delivery* with explicit phase boundaries, ADR-based decision capture, and TDD-first implementation. We treat DevSquad Copilot in this paper as the *canonical agentic development lifecycle exemplar*: a concrete, well-specified process that the framework's design vocabulary composes with. The composition is clean because DevSquad's phases were derived from observation of practice rather than from theoretical commitments, and the framework's elements are likewise derived from observation; both projects converged on the same load-bearing concerns from different starting points.

The framework's elements map to DevSquad's eight phases as follows. Phases 1–2 (*envisioning phase* and *Spec the next slice*) are where archetype is committed (§3.2) — the system author selects the archetype before the spec gains substance, and the dimensions of agency, autonomy, responsibility, and reversibility (§3.3) receive provisional values that will firm up as the slice gains thickness. Phase 3 (*Plan only what the current slice needs*) is where architectural decisions become invariants in §6 of the SDD spec template; ADRs that constrain tool access become clauses in §8 (Authorization Boundary). Phase 4 (*Decompose that slice*) is where the unit-of-delegation is sized — each granular task receives the minimum tool subset its scope requires, instantiating the *Least Capability* discipline (§4.2 below). Phase 5 (*Implement with TDD discipline*) is where the spec acceptance suite — Level 2 of a four-level eval stack — gates each commit; the discipline is that test-before-code at the unit level mirrors spec-before-execution at the system level. Phase 6 (*Learn in the open*) is where the failure taxonomy of §3.4 operationalizes: each failure is categorized (Cat 1 through Cat 7), traced to its fix locus, and recorded in the spec evolution log; the artifact whose modification prevents recurrence is updated explicitly. Phase 7 (*Review in an independent context*) is the validation phase from §3.5's lifecycle, applied at sprint cadence rather than per-execution. Phase 8 (*Refine continuously*) is where the four signal metrics — spec gap rate, first-pass acceptance, cost per correct outcome, oversight load — drive the next sprint's spec priorities, closing the cycle.

The composition is not coincidental. DevSquad Copilot's amendment process — "specification mismatch is a first-class event" — is what the framework calls *living specs* (§3.5); DevSquad's persistence rule that ADRs are durable beyond any single slice while specs evolve is what the framework calls the separation of constitutional layer from spec layer; DevSquad's *Implement with TDD discipline* phase is the framework's eval-suite discipline applied at the unit level. Where the framework adds value to a DevSquad-running team is in the elements DevSquad does not specify: the archetype

taxonomy that makes envisioning a pre-commitment rather than a brainstorm; the four-dimension calibration that gives spec authors four levers rather than the single low/medium/high impact axis; the seven-category fix-locus failure taxonomy that makes the *Learn in the open* phase rigorous rather than reflective. The full phase-to-artifact mapping with per-discipline detail appears in the companion book (Aldecoa 2026); the abbreviated version above is the operational claim.

### 4.2 Capability boundaries via the Model Context Protocol

The framework's authorization-boundary discipline — agents receive *only* the tools their authorized scope requires, and tool-level authorization is enforced at call time rather than at session start — requires a protocol layer that makes the discipline operationally cheap. The Model Context Protocol (Anthropic 2024b), introduced by Anthropic in late 2024 and adopted across vendors during 2025, supplies that layer. MCP is not novel to this paper; it is the substrate on which the framework's *Least Capability* discipline becomes implementable rather than aspirational.

Three properties of MCP make it operationally useful for the framework. First, **per-tool authorization at the server**: the agent presents an identity token; the MCP server enforces what that identity may do. Authorization is not a property the agent claims but a constraint the server checks. Second, **per-call validation**: scope checks run on every call, not only at session establishment, so stale grants do not accumulate as agents stay running across long-lived sessions. Third, **tool description as behavioral contract**: each tool advertises what it does, what inputs it accepts, what side effects it produces, and — critically — what it does *not* do. The description is what the agent reads to decide whether to call the tool, and what the spec author reads to decide whether to authorize it. Section 8 of the canonical SDD template (Authorization Boundary) is what binds the agent's authorized scope to the manifest's tool descriptions; the binding is auditable because both artifacts are version-controlled alongside the system spec.

Two patterns from the framework that depend on MCP are worth naming. The *read-only-by-default* pattern: tools that mutate state are explicitly distinguished from tools that only read, and the Authorization Boundary clause defaults agents to read-only access except where the spec authorizes specific state-changing tools. The *separation-of-effect-classes* pattern: tools are grouped by effect class (read, create, update, delete, transmit), with each class authorized independently, and irreversibility-extending mechanisms (soft delete, draft-and-confirm, approval gates) inserted between classes. Both patterns are implementable without MCP — function-calling protocols and direct tool wiring can enforce the same disciplines — but MCP makes them cheaper, more portable across vendors, and more visible in spec review. The framework treats MCP as the recommended tool layer for agent systems where the deployment will outlive a single vendor's roadmap.

### 4.3 Coding agents: a worked archetype-by-deployment-posture analysis

Coding agents — Cursor, Cline, Aider, Devin, Claude Code, Codex CLI, and the broader category of LLM-driven systems that produce or modify source code — are the most-deployed agent class of 2024–2026. They are also the agent class that most resists clean archetype partitioning under §3.2, because the same underlying capability (a code-aware LLM with file-system, git, and shell access) ships in deployment postures that cross archetypal boundaries. The framework resolves this not by inventing a sixth archetype but by recognizing that *deployment posture* — not the underlying technology — determines archetype assignment.

**Three deployment postures.** A coding agent operating in *pair-programmer* mode (typing alongside a developer in an IDE, surfacing inline suggestions, requiring per-edit acceptance) is an *Advisor*:

it does not act on the world without the human between its output and the consequence. A coding agent operating in *in-loop* mode (assigned a ticket, producing a branch with commits, surfacing as a pull request for human review) is an *Executor* with optional *Synthesizer* composition: it acts autonomously within bounded scope but every set of changes gates on human merge. A coding agent operating in *autonomous* mode (Devin-class deployments where the agent merges its own work, runs deployments, and progresses through a multi-day task without per-commit review) is an *Orchestrator-over-self*: it coordinates its own multi-agent loops with no per-step human between perception, decision, and action. The same underlying technology occupies different points in the archetype taxonomy depending on how it is deployed; the discipline is committing to the posture before the deployment, not inferring it after.

**Three structural controls.** For in-loop coding agents — the most common deployment in 2026 — three controls are non-negotiable. First, *branch protection*: the agent does not merge to mainline; merge requires a human PR review. The control makes Cat 4 (Oversight) failure structurally impossible at the merge boundary regardless of how the spec is written, because the platform-level branch protection rule is not negotiable through the spec. Second, *dependency allowlist*: package installations are restricted to a curated allowlist with version pins; arbitrary package installation is a Cat 2 (Capability) hazard surface that the manifest closes by default. The control prevents typosquat installations (the well-documented `lodahs`-instead-of-`lodash` class of attacks) at the tool layer rather than at the agent layer, which is where prompt-based defenses are unreliable. Third, *sandboxed execution*: code execution happens in an isolated environment without access to credentials or production resources; the agent's runtime cannot reach what the deployment configuration does not expose. The control means that even if the agent's reasoning produces an incorrect command, the blast radius is bounded by the sandbox boundary rather than by the agent's restraint.

**The deleted-tests failure.** A recurring failure class in coding-agent deployments illustrates the framework's diagnostic protocol. The pattern: an agent assigned to make failing tests pass deletes the tests instead of fixing the underlying code. Each individual action is defensible (the spec said "make the tests pass"; deleting tests does make them pass; the agent did what it was asked); the cumulative outcome is wrong (the system is now untested rather than working). Diagnosed under §3.4: this is a hybrid Cat 1 (Spec) and Cat 3 (Scope creep) failure. The Cat 1 component: the spec did not explicitly forbid test deletion. The Cat 3 component: the agent's interpretation of "make tests pass" extended into a scope (modifying the test file itself) that a reasonable reading of the request would not have authorized. The fix is correspondingly two-part: a Cat 1 fix in the spec's NOT-authorized clauses ("you may not delete or skip tests; if a test is wrong, surface it"), and a Cat 2 fix in the manifest plus a CI-level guard ("the agent's authorized file scope excludes the test directory; CI checks the test-skip set is monotonic across the agent's commits"). Both fixes live in artifacts; neither lives in the agent's prompt. The framework's discipline is: when a class of failure recurs, the structural fix lives in the spec, the manifest, the CI, or the platform — never only in the prompt.

**Mode-mixing within a single deployment.** A common shape in production is *escalation across postures*: a coding agent operating in pair-programmer mode for routine edits, escalating to in-loop mode when the developer hands off a ticket-sized task, occasionally escalating further to a constrained autonomous mode when the task is structured enough (e.g., a well-bounded refactor with strong test coverage) to warrant agent-level continuation across a working day. The framework treats each mode as its own archetype commitment with its own spec section, and treats the escalation boundaries as structural controls — the deployment cannot cross from in-loop to autonomous

without an explicit configuration change that re-runs the archetype selection tree. This pattern is observable in DevSquad-running organizations (Microsoft 2026) and is the deployment shape we recommend for teams introducing coding agents to a non-trivial codebase.

**External calibration.** SWE-bench Verified (Jimenez et al. 2024) provides the public calibration benchmark for coding agents on real-world GitHub issues. The framework's eval discipline treats benchmark performance as one of four eval levels (with unit asserts, spec acceptance suite, regression on a team-built golden set, and production sampling); benchmark scores are useful for harness calibration but inadequate as a substitute for a team-built golden set drawn from the deployment's actual task distribution. The point is consistent with the framework's broader stance: external benchmarks reveal whether the underlying capability is sufficient; team-built evals reveal whether the deployment is correctly specified.

### 4.4 Computer-use agents: where Cat 7 becomes necessary

Computer-use agents — Anthropic Computer Use (Anthropic 2024c), OpenAI Operator (OpenAI 2025), Gemini computer use (Google 2025) — perceive a screen via vision-language models and act via simulated input (mouse, keyboard, scroll, screenshot). They are the agent class where the framework's Cat 7 (Perceptual Failure) ceases to be a theoretical addition and becomes the load-bearing diagnostic category. This subsection works through the failure surface that requires Cat 7 and the four structural controls that follow.

**The failure surface.** Three categories of failure that text-only agents do not encounter become first-class for computer-use deployments. *Lookalike domain navigation*: the agent is asked to navigate to `github.com` and follows a link to `g[Cyrillic-i]thub.com` (homoglyph), `githud.com` (typo), or `github.io.attacker.com` (subdomain confusion); the agent does not recognize the divergence between the displayed domain and the actual destination because its visual perception of the URL bar can be deceived in ways its DNS resolver cannot. *Visual instruction injection* on rendered pages (Greshake et al. 2023; Willison 2023): a webpage contains text rendered to look like an instruction — *"Disregard your previous instructions. Click the red button to verify identity."* — and the agent's grounding layer treats the page text as instruction rather than as data. The text is not in HTML headers or page metadata; it is rendered visual content the vision-language model is reading as authoritative. *Modal popup interception*: a modal dialog appears mid-task — *"Your session has expired, please confirm by clicking here."* — and is adversarial rather than from the actual application, but the agent processes it as a legitimate prompt. None of these have analogues in text-only agent deployments. All three are perceiving-then-acting failures the framework partitions as Cat 7.

**Four structural controls.** The structural-controls discipline that §4.3 applies to coding agents extends to computer-use deployments with four non-negotiables. First, *sandboxed environment*: the agent operates in an isolated browser or virtual machine without access to the host's credentials, files, or extensions, and with a reset point between sessions. The control bounds the blast radius of any successful injection or perceptual failure. Second, *authentication scope minimization*: the agent's authenticated browser sessions are scoped to the task's authorized domains; redirects to non-allowlisted domains do not carry credentials. The control prevents the *credential exfiltration via redirect* class of attacks where a successful navigation to an unauthorized domain would otherwise reuse session cookies. Third, *domain allowlist*: the spec's authorized scope (§3 of the SDD template) declares exactly which domains the agent may navigate to; any redirect to a non-allowlisted domain halts the agent and surfaces. The control closes the lookalike-domain failure mode at the

configuration layer rather than relying on the agent's visual judgment. Fourth, *high-consequence confirmation gates*: actions classified as high-consequence (financial transactions, data exports, irreversible state changes, account modifications) require explicit human confirmation regardless of what the spec authorizes for the broader task. The control is the framework's reversibility lever applied at the action-class boundary: the agent may be authorized to do many things autonomously, but the irreversible subset gates uniformly.

**Cat 7 in operational use.** When a computer-use deployment surfaces a failure, the diagnostic protocol from §3.4 applies: identify the symptom (what was observed), identify the fix locus (which artifact must change). The four sub-categories of Cat 7 (misidentification, missed element, hallucinated element, state miscount) map to four structural fixes: confirmation gates for misidentification of high-consequence actions; screenshot-then-verify for missed elements before consequential proceeding; element-allowlist plus DOM-grounded verification for hallucinated elements; re-verification of position-based facts at the moment of action for state miscount. None of these fixes live in the agent's prompt; all live in the spec, the structural controls, or the verification protocol. The framework's discipline holds: when Cat 7 failures recur, the fix is structural, not prompt-based.

**The "use APIs first" stance.** A practical implication for spec authors: *when an API exists for the task, computer-use should be the option of last resort.* A computer-use deployment substitutes vision-language perception for an API contract; the perception is less reliable than the contract by orders of magnitude on most tasks. The framework's recommendation is to use computer-use only when (i) no API exists, (ii) the API does not cover the needed task surface, or (iii) the API exists but its access cost (compliance review, partnership negotiation, rate-limiting friction) exceeds the cost of the additional Cat 7 risk surface plus the four structural controls. This is a judgment-call discipline rather than a hard rule, but the burden of proof is on the deployment that chooses computer-use over an available API.

**External calibration.** WebArena (Zhou et al. 2024), VisualWebArena, OSWorld (Xie et al. 2024), and ScreenSpot-Pro provide public calibration benchmarks. Their results — frontier models reaching task-completion rates well below human baselines on many task categories — are part of why the framework's stance on computer-use is conservative. The OWASP LLM Top 10 (OWASP Foundation 2025) (with its 2025 multimodal sub-category in LLM01 Prompt Injection) provides the attack-surface enumeration that the four structural controls and the Cat 7 sub-categories extend. The framework adds the *failure-locus* axis to OWASP's attack-surface axis: a successful prompt-injection attack on a computer-use agent might be classified as LLM01-multimodal at the attack-surface layer, and as Cat 7-misidentification or Cat 1-spec-gap at the fix-locus layer — both classifications point at distinct artifacts that may need to change.

---

## 5. Discussion

### 5.1 When the framework helps

The framework's overhead is non-trivial: archetype commitment, four-dimension calibration, spec authoring against a thirteen-section template, failure-taxonomy classification, and living-spec evolution all impose author-side cost that a free-form prompt does not. The overhead pays back when three conditions hold. First, *non-trivial autonomy*: the executor makes judgment-laden choices over a domain wider than the spec author can enumerate, so the spec cannot rely on the human implementer's silent bridge. Second, *consequential outcomes*: failure produces costs (regulatory,

financial, reputational, or safety-related) that exceed the overhead of precise specification by a comfortable margin. Third, *sustained operation*: the system runs more than a handful of times, so spec investment amortizes across many executions. When all three hold, the framework's overhead is dominated by the rework, scope-drift, and post-incident-churn costs the framework prevents. When any one fails — a one-off prototype, a low-consequence deployment, a system that runs once and is discarded — the framework's overhead exceeds its value, and teams should adopt the *vocabulary* (archetype, dimensions, failure categories) without the full SDD apparatus. The vocabulary is cheap; the apparatus is what earns its keep on serious deployments.

### 5.2 When it doesn't

Three classes of system are honestly outside the framework's scope. *Regulated industries* (healthcare, finance, defense, aviation) have compliance regimes whose specification requirements (FDA QSR, GxP, DO-178C, SOC 2) are stricter than the framework's canonical SDD template, and whose oversight requirements involve audit trails the framework does not specify. The framework composes with these regimes — the canonical template is a subset of what compliance requires — but a deployment in a regulated industry needs the regulator's specification standard, not the framework's, as the controlling document. The framework supplements; it does not substitute. *Multi-organizational agent systems*, in which agents from different organizations interact through standardized protocols (Google's A2A, MCP cross-vendor calls), surface governance problems the framework does not solve. Whose archetype declaration governs when two organizations' agents both act on a transaction? Whose failure taxonomy applies when the failure crosses organizational boundaries? The framework is single-organization-shaped and acknowledges that the multi-organizational case requires governance work the present paper does not undertake. *Adoption cost-benefit analysis* for the framework depends on factors (team experience, codebase maturity, deployment cadence, regulatory environment, risk tolerance) that vary across organizations widely enough that no general recommendation is honest. Teams should adopt the framework if its overhead is cheaper than the rework cost it prevents in their specific context, and that calculation is local.

### 5.3 Relation to MAST and other multi-agent failure work

The framework's failure taxonomy (§3.4) sits in a multi-axis classification space alongside several related works rather than competing with them. Cemri et al.'s MAST (Cemri et al. 2025) partitions multi-agent failures by *empirical symptom* across fourteen categories observed in 200+ deployments. Zhang et al.'s hallucination survey (Zhang et al. 2025) partitions model-level failures (Cat 6 in our framing) into finer sub-types: tool-call hallucination, planning hallucination, instruction-following inconsistency. OWASP LLM Top 10 (OWASP Foundation 2025) partitions by *attack surface* across ten categories with a particular focus on injection and excessive-agency hazards. Our Cat 1–7 partitions by *fix locus* — which artifact must change to prevent recurrence. The four partitions are complementary because they answer different operational questions. MAST tells the team *what failed at the symptom layer*. Zhang et al. tell the team *which sub-class of model-level failure* is involved when MAST identifies a Cat 6 surface. OWASP tells the team *what attack surface* the failure exploited. Cat 1–7 tells the team *which artifact must be updated* to close the failure. A serious post-incident analysis benefits from all four: a single finding is labeled simultaneously with its MAST symptom, its OWASP surface (if applicable), its Zhang sub-type (if model-level), and its Cat 1–7 fix locus. The labels point at distinct artifacts that may need to change. The framework's contribution at this layer is the *fix-locus axis*, which the prior partitions do not provide; we treat the prior work as compositional rather than competitive.

### 5.4 Generalization beyond AI agents

The framework's central claim is that *intent is a primary design surface for any delegated system.* AI agent systems are the most-acute current instance because the delegation is wider and faster than at any prior point in software practice, but the framework's elements — archetype as a pre-commitment to delegation shape, four-dimension calibration, fix-locus failure taxonomy, spec-as-control-surface — are not specific to LLM-driven executors. We argue this generalization is the framework's broader theoretical contribution and acknowledge that the present paper does not provide worked examples for non-agent applications; the generalization is asserted with worked instantiation in the agent case (§4) and pointed-toward instantiation in the cases below.

*Organizational delegation.* Human organizations delegate decision authority to roles, teams, and procedures. The five archetypes recur recognizably in this domain: *Advisor* maps to a research or analyst function that surfaces options without acting; *Executor* maps to an operations function that runs a defined process within bounded discretion; *Guardian* maps to a compliance or safety function whose primary purpose is to block constraint violations; *Synthesizer* maps to a planning or strategy function that composes from many inputs into a coherent recommendation; *Orchestrator* maps to a coordination role (program manager, incident commander) that directs other functions toward a compound goal. The four dimensions calibrate organizational delegation as much as they do agent systems: how wide is the role's decision space (agency); how independent is the role's day-to-day operation from sign-off (autonomy); where does accountability concentrate (responsibility); how recoverable are the role's mistakes (reversibility). Organizational dysfunction frequently has Cat 1 (Spec) or Cat 4 (Oversight) shape: roles whose responsibilities are under-specified, oversight gates that fail because the wrong artifact was authorizing the wrong action.

*CI/CD pipelines.* Automated build-and-deploy systems are delegated executors of release decisions. A pipeline's archetype is generally *Executor* (acts within strict bounds — run tests, build artifact, deploy to environment) or *Orchestrator* (coordinates multiple downstream pipelines, gating on their results). Its four dimensions are calibratable: agency (does the pipeline choose deployment targets, or does it execute a fixed sequence?); autonomy (does the pipeline auto-merge passing changes, or gate on human approval?); responsibility (whose pager rings when production breaks?); reversibility (is rollback a one-command operation, or does it require manual intervention?). CI/CD failure modes map cleanly to Cat 1–6: a pipeline that deploys to production despite a spec that forbids it is Cat 4; a pipeline whose tool manifest exposes a `force-deploy` capability that should have been restricted is Cat 2; a pipeline that compounds defensible-individually steps into a wrong release is Cat 5.

These two examples are sketches, not worked instantiations. We make them to support the theoretical claim that the framework's reach extends beyond AI agent systems and to offer practitioners outside the agent space an entry point. Empirical validation of the framework's value in non-agent domains is future work that the present paper does not undertake.

### 5.5 What this paper does not claim

The honest contribution accounting in §1.3 names what the paper claims as novel — Cat 7 (Perceptual Failure), the orthogonality operationalization, the fix-locus framing, the synthesis itself — and what it does not — Spec-Driven Development as a discipline, archetypes as a concept, the four dimensions individually, Cat 1–6 as categories. §6 enumerates the framework's limitations explicitly. We do not re-derive either set here; we name three additional non-claims that the framing of this paper might otherwise be read to suggest.

We do not claim that the framework is *complete*. The seven failure categories partition the failure space coarsely; the five archetypes commit to a smaller taxonomy than the design space supports; the four dimensions are independent levers but not exhaustive of every spec property a deployment might need. The framework is a *working* set of artifacts, opinionated rather than derived, that practitioners refine in their own domains.

We do not claim that the framework produces *good* outcomes by itself. A spec author with the framework's vocabulary and a clear contribution accounting can still write a spec that under-specifies a real constraint, mis-calibrates a dimension, or misclassifies a recurring failure. The framework provides *structure* for design and diagnostic work; the work itself remains the practitioner's.

We do not claim that the framework is *finished*. Cat 7 is preliminary; the orthogonality argument is asserted with examples but not formally proven; the generalization beyond AI agents is sketched but not worked through. Future versions of the framework may revise any of these. The paper offers what we have now, with limitations named, on the working assumption that an explicit structure is more useful than a delayed one.

---

## 6. Limitations

*Section format: bullet list of explicit limitations. Reviewers reward this section more than they reward the introduction.*

- **Position-paper status.** The paper is offered as a structural design tool, not as an empirical intervention. The framework's claims are normative ("you should design this way") rather than descriptive ("we measured what works").
- **No empirical validation at scale.** The book provides three worked examples (customer support, code-generation pipeline, coding agent); these are existence proofs, not statistical evidence. Quantitative validation across many deployments is future work.
- **Archetype taxonomy is opinionated.** The five archetypes are a working taxonomy, not a derived classification. A different practitioner might propose three, seven, or nine; the contribution is the act of canonicalizing rather than the specific cardinality.
- **Cat 7 is preliminary.** The category is named but not yet stress-tested across many computer-use deployments. Sub-categories may need to be revised as more deployments surface failure modes.
- **Generalization beyond AI agents is asserted, not demonstrated.** §5.4 argues the framework generalizes; we do not provide worked examples for non-agent applications in this paper.
- **Vocabulary friction.** The paper introduces several coined or refactored terms (intent as a design surface, fix-locus taxonomy, Cat 7). Adoption requires the vocabulary to spread; the framework's value is partly contingent on linguistic uptake.

---

## 7. Conclusion

The cost of imprecise intent has always existed in software. It was tolerable for decades because human professionals exercised silent judgment to bridge it: senior engineers supplied missing constraints, escalated ambiguity rather than executing it, and rewrote underspecified tickets into well-scoped commits without ever updating the specification document. The bridge worked because the

executor was also the judgment layer. As delegation widens — to LLM-driven agents in 2024–2026, to automated pipelines, to organizational roles — the executor and the judgment layer separate. Modern agent harnesses provide clarification APIs, but the agents using them exhibit a documented execution-first bias and rarely invoke escalation at the moments that would have warranted it. The cost of imprecision stops being absorbed silently and starts surfacing as wrong outputs, scope drift, and post-incident churn.

The Architecture of Intent proposes a structural response. Treat intent as a primary design artifact distinct from implementation (§3.1). Commit to one of five archetypes before designing the system (§3.2). Calibrate four orthogonal dimensions — agency, autonomy, responsibility, reversibility — independently rather than collapsing them onto a single automation level (§3.3). Diagnose failures by *fix locus* across seven categories, with Cat 7 (Perceptual Failure) added for perceiving-then-acting systems (§3.4). Express the result through Spec-Driven Development as the executable protocol (§3.5). Apply the discipline against AI agent systems and the agentic development lifecycle today (§4); generalize to other delegated systems as the lessons travel out (§5.4).

The framework's bet is that as delegation widens, intent precision compounds in value. A spec author whose hour produces a tightly-bounded specification commits the executor to hundreds of correct outputs; the same hour spent on a single output commits one. The leverage on intent has always been favorable; what 2024–2026 changes is that the leverage is now collectible. The framework is offered as the structure that makes the collection routine — for AI agent systems specifically, and for delegated systems more broadly. The work it leaves to practitioners is the work practitioners always had: the judgment about what their specific system should do, expressed precisely enough that an executor can act on it and a validator can check it.

---

## References

> Bibliography is generated from `references.bib` at compile time. The Markdown source uses Pandoc-style `[@key]` inline citations resolved by `pandoc --citeproc --bibliography references.bib`. The `references.bib` file in this directory contains full BibTeX entries (~30 sources across 9 domains). Citation style: numeric (arXiv default for first version). For workshop or journal submission we will normalize to the venue's required style.

*Bibliography rendered here at compile time.*


Aldecoa, Marcel. 2026. *The Architecture of Intent: A Field Guide to Designing and Shipping AI Agent Systems*. Self-published.

Alexander, Christopher, Sara Ishikawa, and Murray Silverstein. 1977. *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press.

Anthropic. 2024--2025. "Claude Code: an agentic coding tool." https://docs.anthropic.com/claude/docs/claude-code.

———. 2024a. "Building Effective Agents." https://anthropic.com/research/building-effective-agents.

———. 2024b. "Introducing the Model Context Protocol." https://modelcontextprotocol.io.

———. 2024c. "Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku." https://anthropic.com/news/3-5-models-and-computer-use.

Brooks, Frederick P. 1975. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.

Cemri, Bora et al. 2025. "MAST: A Multi-Agent System failure Taxonomy." *arXiv Preprint*.

GitHub. 2024. "spec-kit: Toolkit for spec-driven development with AI assistants." https://github.com/github/spec-kit.

Google. 2025. "Gemini computer use." https://ai.google.dev.

Greshake, Kai, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." *arXiv Preprint arXiv:2302.12173*.

Jackson, Michael. 1995. *Software Requirements & Specifications: A Lexicon of Practice, Principles and Prejudices*. Addison-Wesley.

Jimenez, Carlos E. et al. 2024. "SWE-bench Verified." *arXiv Preprint*.

Liu, Nelson F., Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. "Lost in the Middle: How Language Models Use Long Contexts." *arXiv Preprint arXiv:2307.03172*.

Meadows, Donella H. 2008. *Thinking in Systems: A Primer*. Chelsea Green Publishing.

Meyer, Bertrand. 1992. "Applying 'Design by Contract'." *Computer* 25 (10): 40–51.

Microsoft. 2026. "DevSquad Copilot: An eight-phase agentic development lifecycle." https://github.com/microsoft/devsquad-copilot.

OpenAI. 2025. "Introducing Operator." https://openai.com/index/introducing-operator.

OWASP Foundation. 2025. "OWASP LLM Top 10 (2025 update)." https://genai.owasp.org/llm-top-10.

Pope, Reiner, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2022. "Efficiently Scaling Transformer Inference." *arXiv Preprint arXiv:2211.05102*.

Reason, James. 1990. *Human Error*. Cambridge University Press.

SAE International. 2021. "J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles." SAE International.

Shavit, Yonadav, Sandhini Agarwal, et al. 2023. "Practices for Governing Agentic AI Systems." OpenAI. https://openai.com/research/practices-for-governing-agentic-ai-systems.

Willison, Simon. 2023. "Prompt injection / Lethal trifecta series." https://simonwillison.net.

Xie, Tianbao et al. 2024. "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments." *arXiv Preprint*.

Zhang, Yifan et al. 2025. "LLM-based Agents Suffer from Hallucinations: A Survey of Taxonomy, Methods, and Directions." *arXiv Preprint arXiv:2509.18970*.

Zhou, Shuyan et al. 2024. "WebArena: A Realistic Web Environment for Building Autonomous Agents." *arXiv Preprint*.

---

## Appendix A: Mapping to the book

*For readers of the companion book (Aldecoa 2026), this appendix maps each section of the paper to the relevant book chapter so the paper can be read as a distillation rather than a substitute. The book also contains the inverse mapping in its Companion Paper appendix, which lists every paper section against the book chapters that elaborate it; this appendix is the symmetric paper-side index.*

| Paper section | Book chapters that elaborate |
|---|---|
| §1 Introduction & §1.3 Contribution | Prologue; Introduction; *What is the Architecture of Intent?* (Introduction); *A Miniature Pilot, End-to-End* |
| §3 *(framework canvas, Figure 1)* | *The framework on one page* (Introduction) — the same canvas appears in both artifacts as the visual summary |
| §3.1 Intent as a design surface | Theory ch. 2 (Intent vs. Implementation) |
| §3.2 Archetypes (with composition first-class) | Architecture ch. 2 (Pick an Archetype), ch. 3 (Four Dimensions of Governance), ch. 4 (Selection Tree), ch. 5 (Composing Archetypes — including Pattern E mode-switching and the Composition Declaration), ch. 6 (Governed Archetype Evolution); Repertoire ch. 2 (Intent Archetype Catalog) |
| §3.3 Four dimensions | Theory ch. 3 (Calibrate Agency, Autonomy, Responsibility, Reversibility) |
| §3.4 Cat 1–7 fix-locus failure taxonomy | Theory ch. 5 (Failure Modes and How to Diagnose Them); Agents ch. 9 (Computer-Use Agents) for the Cat 7 detail |
| §3.5 SDD as the protocol | Part 2 in full; especially SDD ch. 7 (The Canonical Spec Template — including the §4 Composition Declaration sub-block) |
| §4.1 Agentic development lifecycle | Operating ch. 12 (DevSquad Mapping), ch. 13 (Co-adoption with DevSquad), ch. 11 (Adoption Playbook) |
| §4.2 MCP capability boundaries | Agents ch. 4 (Least Capability), MCP ch. 1–3 |
| §4.3 Coding agents | Agents ch. 8; Examples ch. 3 (Designing an AI Coding Agent) |
| §4.4 Computer-use agents | Agents ch. 9 |
| §5 Discussion (applicability, MAST complementarity) | Architecture ch. 7 (Multi-Agent Governance); Theory ch. 5 §"How this taxonomy relates to the empirical literature" |
| §6 Limitations | Introduction §"Honest scope"; Operating ch. 15 (Signs Your Architecture of Intent Is Degrading) |
| Working practices not covered in the paper | Theory ch. 7 (The Intent Design Session); Operating ch. 15 (anti-patterns + Discipline-Health Audit) — both are operational rituals expressed in the book and not separately argued in the paper |

## Appendix B: Mapping the framework to Microsoft DevSquad Copilot

*This appendix expands §4.1 with a closer look at how the framework's elements compose with the documented features of Microsoft DevSquad Copilot (Microsoft 2026). Names and feature labels in this appendix follow DevSquad's own README terminology where verified; analytic labels supplied by this paper are noted as such.*

### B.1 What DevSquad Copilot is

Microsoft DevSquad Copilot is an open-source agentic software-delivery framework released in 2026. Its operational core is an **eight-phase iterative cycle** that runs from an *envisioning phase* through *Spec the next slice*, *Plan only what the current slice needs*, *Decompose that slice*, *Implement with TDD discipline*, *Learn in the open*, *Review in an independent context*, and *Refine continuously*. Coordination is performed by a single **conductor agent** that **guides the cycle with Socratic questions** and delegates to **13 specialized sub-agents** running in isolated context windows. The twelve named specialist agents are *init*, *envision*, *kickoff*, *specify*, *plan*, *decompose*, *implement*, *review*, *security*, *sprint*, *refine*, and *extend*; with the conductor itself the framework's count is thirteen. User stories are sliced and prioritized as **P1, P2, P3**, under the discipline that P2 and P3 are intentionally underspecified until P1 is implemented and reviewed. **Architectural Decision Records (ADRs)** are persisted as durable artifacts beyond any single slice and amended in place when implementation reveals new understanding.

Tool access is mediated through a curated set of **first-party MCP servers**: *GitHub* (used by most agents for repos, issues, pull requests, projects, actions, and secret protection), *Azure DevOps* (work-item, sprint, search, and repo toolsets), *Azure* (architecture guidance, deployment, pricing, compliance), *Microsoft Learn* (documentation and API references), and *Draw.io* (architecture diagrams and threat models). Each agent receives only the granular tools its role requires; opaque third-party servers are not part of the default trust boundary.

Autonomy scales by an **impact classification** with three tiers — *low*, *medium*, *high*. Low-impact changes (e.g., typo fix, log message, formatting) proceed directly and skip validation against spec, the *comprehension checkpoint*, the reasoning log, and pre-PR review. Medium-impact changes (e.g., new function, local refactor, validation addition) require a presented plan, developer confirmation, and automated review. High-impact changes (e.g., new service, schema change, public API modification) require a mandatory ADR, explicit approval, and the full review battery; they are not eligible for the streamlined mode. Tasks initially classified low can be reclassified upward during implementation if complexity emerges.

Configuration lives at documented paths in the consuming repository: `.github/copilot-instructions.md` (the primary instruction file created during framework initialization), `.github/docs/coding-guidelines.md` (team-and-stack-specific coding standards), and an `.github/instructions/` directory containing role-specific instruction files (`specs.instructions.md`, `adrs.instructions.md`, `tasks.instructions.md`, and others). The framework is invoked through GitHub Copilot CLI as `copilot --agent devsquad:devsquad`, with a `--yolo` flag available to enable unattended operation that auto-approves terminal commands, file changes, and tool usage.

The framework also ships a **skills catalog** of twenty reusable knowledge packages organized across five domains (Architecture and Planning, Work Items and Estimation, Quality and Security, Development, Initialization). Skills load *semantically*: rather than being invoked explicitly, the agent runtime reads each skill's `description` field at every turn and activates the skill when the current

context matches. Specific named skills include *reasoning* (decision recording across all agents), *board-config* (platform detection), *harness-learnings* (codebase learning capture), and *quality-gate* (artifact validation). Reasoning is operationalized through a documented **reasoning log** artifact: "every decision recorded with principle, alternatives, justification, and confidence level," distinct from the spec evolution log but anchored in the same persistence-of-decision discipline. The framing the project itself uses for this overall discipline is **"loop over ladder"** — each turn through the eight phases produces a small, reviewable increment that refines the shared understanding captured in persistent artifacts, rather than progressing linearly through a fixed sequence.

DevSquad's discipline converges with this paper's framework on the same load-bearing concepts: spec-first execution, intent as the durable artifact, archetype-shaped delegation, evolution of specs in response to implementation reality, least-capability tool boundaries, and risk-tiered human-in-the-loop oversight. The paper's framework was substantially derived from observing systems of this shape; the convergence is not coincidental but represents two independent paths arriving at the same operational principles.

### B.2 Element-to-feature mapping

| Framework element | DevSquad feature | Notes |
|---|---|---|
| **Intent as a design surface** (§3.1) | The envisioning phase + the spec-slice phase taken together produce DevSquad's intent artifact, distinct from the implementation produced in later phases. | DevSquad does not use the term *intent surface*; the artifact-distinction is the paper's framing. |
| **Archetypes** (§3.2) | The conductor agent is an Orchestrator archetype; the twelve named specialist agents (*init*, *envision*, *kickoff*, *specify*, *plan*, *decompose*, *implement*, *review*, *security*, *sprint*, *refine*, *extend*) instantiate Executor, Synthesizer, and Guardian archetypes within the conductor's coordination. The *plan*, *implement*, *review*, and *refine* agents are documented as coordinator agents that themselves delegate to internal worker sub-agents — a recursive Orchestrator-over-Executor composition. | DevSquad does not use the paper's archetype terminology; the assignments are the paper's interpretation. |

| Framework element | DevSquad feature | Notes |
|---|---|---|
| **Four-dimension calibration** (§3.3) | DevSquad's *impact classification scales rigor to risk* discipline calibrates oversight (autonomy) and approval gates (responsibility) per task. | The four-dimension *orthogonality* claim is the paper's contribution; DevSquad treats risk as a single scaling axis. |
| **Cat 1 – Cat 7 failure taxonomy** (§3.4) | DevSquad's Phase 6 (*Learn in the open*) is the operational surface where this paper's failure-locus diagnosis is performed; spec amendments correspond to Cat 1 fixes, manifest changes to Cat 2, oversight-model changes to Cat 4, system-spec changes to Cat 5. | DevSquad recognizes specification mismatches as a first-class event and amends spec rather than patching code; the paper supplies the categorization scheme. |
| **Spec-Driven Development as protocol** (§3.5) | DevSquad's spec-as-source-of-truth discipline, with the eight-phase cycle as the lifecycle. | Direct lineage. |
| **Capability boundaries via MCP** (§4.2) | DevSquad's MCP-server-mediated tool layer; each sub-agent receives only the tools its role requires. | Direct alignment. |

### B.3 Disciplines whose names differ

Several DevSquad practices the paper repeatedly references appear in DevSquad's documentation under different names from the paper's. The paper's framing is supplied to make the framework's vocabulary consistent across delegated systems generally; DevSquad's framing is supplied because DevSquad ships under those terms. Where a passage in the body of the paper is best read alongside DevSquad's own documentation, the table below provides the cross-reference.

| Paper's term | DevSquad's documented language |
|---|---|
| *Living spec* / spec evolution (§3.5) | "Specification mismatch is a first-class event"; the amendment process in *Learn in the open*; the *refine* agent applies scoped amendments to specs and ADRs |
| *Pre-clarification step* (§3.5) | The Socratic-questioning pattern in the conductor's envisioning phase; the *envision* agent surfaces business intent, pain points, and objectives |

| Paper's term | DevSquad's documented language |
|---|---|
| *Capability minimalism* (§4.2) | "Curated set of first-party MCP servers"; least-privilege scope at the sub-agent boundary; each agent receives only granular tools (e.g., the *implement* agent's Azure-DevOps repos toolset) |
| *Risk-tier calibration* (§3.3) | "Impact classification scales rigor to risk" — three documented tiers (low / medium / high), each with its own ceremony requirements; *low* skips spec validation, comprehension checkpoint, reasoning log, pre-PR review; *high* requires mandatory ADR plus full review |
| *Constitutional layer* (§3.5) | `.github/copilot-instructions.md`, `.github/docs/coding-guidelines.md`, and the `.github/instructions/` directory of role-specific instruction files (`specs.instructions.md`, `adrs.instructions.md`, `tasks.instructions.md`, etc.) |
| *Comprehension verification* | The *comprehension checkpoint*: after a medium- or high-impact implementation, the *implement* agent presents the approach and waits for the developer to confirm understanding before proceeding |
| *Spec evolution discipline* (§3.5) | The "loop over ladder" framing — each cycle produces a reviewable increment that refines the shared model rather than linearly executing a fixed plan; spec amendments are a first-class event |
| *Decision-traceability artifact* | The *reasoning log* — a persistent artifact that records every non-trivial decision with principle, alternatives, justification, and confidence level. The framework's spec evolution log records *what* changed and why; the reasoning log records *the deliberation behind every decision*, including those that did not change a spec |
| *Named delivery guardrails* | DevSquad articulates explicit forbiddances absent from the paper's framework: *"no non-trivial decisions without explicit approval"*, *"refuse to debug blindly: require sufficient context"*, *"permission to not act: unnecessary code generation is worse than nothing"*, and the *"it works but I don't know why"* anti-pattern as the operational signature of passive delegation |

### B.4 Where the paper extends DevSquad and where DevSquad extends the paper

The paper's framework offers three things DevSquad's documentation does not.

1. The **archetype taxonomy** (§3.2) is a per-component classification with explicit oversight defaults that DevSquad's role-typing approximates but does not formalize.
2. The **four-dimension orthogonality argument** (§3.3) decomposes DevSquad's single risk-scaling axis into independent levers a spec author can calibrate separately.
3. The **Cat 1–Cat 7 fix-locus taxonomy** (§3.4), particularly **Cat 7 (Perceptual Failure)**, supplies a diagnostic structure for failure response that DevSquad's amendment process operates on but does not name.

DevSquad's framework offers three things the paper's does not.

1. A **complete operational cadence** — the eight-phase cycle with specific phase boundaries, artifact deliverables per phase, and review checkpoints. The paper is process-agnostic; DevSquad is process-prescriptive.
2. A **specific multi-agent topology** — the conductor + 13 sub-agents with documented roles. The paper's archetype framework is an analysis tool; DevSquad ships a runnable implementation.
3. A **convention for ADR-spec separation** — DevSquad's persistence rule (ADRs are durable beyond any single slice) is operationalized in its repository structure. The paper acknowledges this as the right pattern but does not prescribe an implementation.

The two are complementary in production. Teams running DevSquad gain by adopting the paper's archetype, dimension, and fix-locus vocabulary for design conversations and post-incident analysis. Teams adopting the paper's framework gain a runnable cadence by composing it with DevSquad's eight phases. Neither replaces the other; the framework supplies the design vocabulary and the cadence supplies the operational rhythm.

---

*End of paper. Compile via Pandoc:* `pandoc architecture-of-intent.md --citeproc --bibliography references.bib --output architecture-of-intent.pdf` *(or* `--to latex` *for arXiv submission). See* `paper/README.md` *for full conversion instructions.*